Taller 3

Fecha: Marzo de 2026

Sistema de habilidades a evaluar: Usar los conceptos del modelo relacional de las bases de datos, de los lenguajes DDL (*Data Definition Language*), DML (*Data Management Language*) del SQL, y de los lenguajes procedimentales como T-SQL (*Transact - Structured Query Language*) para la gestión de bases de datos en motores de bases de datos.

NOTAS:

- Este taller se debe hacer con carácter de repaso de las competencias básicas en Bases de Datos.
- Se entregan ejercicios resueltos como ejemplo para el desarrollo de los demás.

Programación en los motores de bases de datos

La programación en un motor de base de datos permite la capacidad de escribir y ejecutar código o scripts directamente en el mismo motor para poder ejecutar operaciones específicas. Esto implica la creación de **procedimientos almacenados**, **funciones**, **disparadores** (*triggers*) u otros artefactos de programación que se ejecutan en el servidor de la base de datos en respuesta a eventos o solicitudes.

Esta forma de programación permite llevar a cabo tareas como la manipulación y el procesamiento de datos dentro de la base de datos, la automatización de procesos de negocio y la implementación de lógica de aplicación directamente en el backend, lo que puede mejorar el rendimiento y la seguridad de la aplicación. Ejemplos comunes incluyen el uso de SQL-PL (*DB2*), PL/SQL (*Oracle)*, T-SQL (*Sql Server*), SQL/PSM (*MySQL)* entre otros lenguajes específicos del motor de base de datos para lograr una funcionalidad específica en la base de datos.

El SQL (*Structured Query Language*) se considera un lenguaje de programación declarativo. La programación declarativa es un paradigma de programación en el que se especifica el "qué" se debe hacer en lugar del "cómo" se debe hacer. En SQL, los usuarios y desarrolladores declaran las operaciones que desean realizar en una base de datos, como consultas, inserciones, actualizaciones o eliminaciones, sin especificar la lógica detallada de cómo se deben llevar a cabo esas operaciones.

Puesta a punto de una base de datos

Para poder realizarla, es preciso en primer lugar dominar los siguientes conceptos:

- El **modelo de datos relacional** representa una base de datos como un conjunto de tablas.

  Cada **tabla** tiene un número de columnas las cuales tienen nombres únicos. Estas columnas corresponden a los atributos o **Campos**. Las tablas a su vez, presentan renglones con los valores de los atributos para cada una de las instancias (**Registros**). Veamos un ejemplo con la tabla para almacenar las *Monedas*:

Campos

Registros

Id	Moneda	Sigla	Simbolo	Emisor	Imagen
2	Afgani	AFN	NULL	NULL	NULL
91	Ariary malgache	MGA	NULL	NULL	NULL
138	Baht	THB	NULL	NULL	NULL
111	Balboa	PAB	NULL	NULL	NULL
48	Birr etíope	ETB	NULL	NULL	NULL
154	Bolívar	VEF	NULL	NULL	NULL
155	Bolívar soberano	VES7	NULL	NULL	NULL
19	Boliviano	BOB	NULL	NULL	NULL
54	Cedi ghanés	GHS	NULL	NULL	NULL
74	Chelín keniano	KES	NULL	NULL	NULL
131	Chelín somalí	SOS	NULL	NULL	NULL
146	Chelín tanzano	TZS	NULL	NULL	NULL
148	Chelín ugandés	UGX	NULL	NULL	NULL
37	Colón costarrice...	CRC	NULL	NULL	NULL
135	Colon Salvador...	SVC	NULL	NULL	NULL
106	Córdoba	NIO	NULL	NULL	NULL

Existe una correspondencia directa entre el concepto de tabla y el concepto matemático de una relación puesto que cada columna de una tabla representa una relación entre un conjunto de valores. De esta manera, una tabla es un conjunto de relaciones, lo cual se denota así:

$$\underset{i=1}{\overset{n}{X}} D_i$$

Donde:

- *n* es el número de columnas de la tabla
- D_i es el conjunto de valores para la columna *i*.

A cada línea de una tabla se le conoce también como **tupla**. Para indicar un valor de un atributo en una tupla determinada se utiliza la notación:

t_i ***[atributo]***

Donde

- *i* es el número de la tupla
- *atributo* es el nombre del atributo al cual corresponden al valor

El esquema de una tabla se denotaría así:

```
Nombre tabla (atributo: Dominio, …)
```

Por ejemplo, para la tabla anterior, su esquema sería :

```
Moneda (Id: entero,
        Moneda : cadena,
        Sigla : cadena,
        Simbolo: cadena,
        Emisor: cadena,
        Imagen: bytes)
```

y una variable de tupla de la tabla sería:

```
t4 (Moneda) = 'Balboa'
```

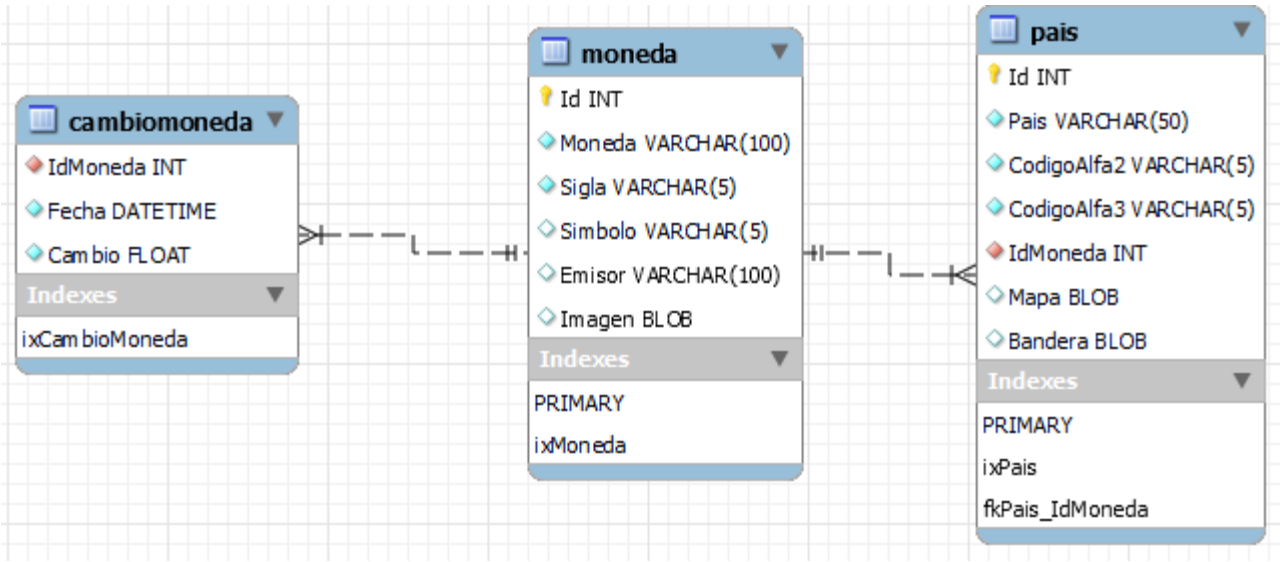
Lo cual quiere decir que el atributo ***Moneda*** en la tupla ***4*** equivale a la cadena '*Balboa'*.

El usuario de un sistema de base de datos puede realizar consultas de estas tablas, inserción de nuevas tupas o actualización de las ya existentes. La expresión de dichas operaciones se realiza mediante unos lenguajes conocidos como **Lenguajes de Consulta Relacional**, como lo es el SQL que se detalla más adelante.

- Actualmente, todos los sistemas de gestión de bases de datos modernos almacenan y tratan la información utilizando el **Modelo relacional**. El nombre relacional procede del hecho de que cada registro de la base de datos contiene información relacionada con un tema y sólo con ese tema. Además, los datos de dos clases de información (tal como países y monedas) pueden ser manipulados como una única entidad basada en los valores de los datos relacionados. Por ejemplo, sería una redundancia almacenar la información sobre el nombre y la sigla de una moneda con todos los países donde operan. De modo que, en un sistema relacional, la información sobre los países contiene algún dato que puede ser utilizado para enlazar cada país con su respectiva moneda. En un **Sistema de Gestión de Base de Datos Relacionales** (*DBMS*) el sistema trata todos los datos en tablas. Cada **tabla** almacena información sobre un tema (como por ejemplo los autores) y dispone de unas columnas (**campos**) que contienen los diferentes tipos de información sobre ese tema (como por ejemplo el nombre) y de unas filas (**registros**) que describen todos los atributos de una única instancia del tema (por ejemplo, los datos de la moneda "Euro"). Incluso cuando se consulta la base de datos (**consulta**) para buscar información procedente de una o más tablas, el resultado es siempre algo semejante a otra tabla.

  La forma como la información de una tabla está relacionada con la información de otra tabla se llamada **Relación**. Por ejemplo, las *monedas* tienen una relación uno a muchos con los *países* ya que una moneda puede ser oficial en muchos países y un país sólo tiene una moneda oficial. Los *cambios de moneda* tienen una relación varios a una con las monedas ya que una moneda puede tener registros de varios cambios (uno por día) y un cambio sólo corresponde a una moneda.

En resumen, el modelo de datos relacional representa una base de datos como un conjunto de tablas y unas relaciones entre las tablas. La siguiente gráfica ilustra el **Diagrama Relacional** obtenido en el DBMS *MySQL* para la base de datos que se está trabajando en el ejercicio:



Para poder crear dicha base de datos es preciso comprender el siguiente tema.

- El **Lenguaje de Definición de Datos** (en inglés DDL, ***Data Definition Language***) es un lenguaje especial que permite la especificación de un esquema de una base de datos por medio de una serie de definiciones.

  El resultado de la compilación de las proposiciones en DDL es un conjunto de tablas que se almacenan en un archivo especial conocido como Diccionario de Datos.

  Por ejemplo, para la base de datos del ejercicio, las siguientes serían las instrucciones en lenguaje DDL para crear la base de datos en el motor ***Sql Server***:

```
CREATE DATABASE Monedas
GO

USE Monedas
GO

/* Crear tabla MONEDA */
CREATE TABLE Moneda(
 Id int IDENTITY(1,1) NOT NULL,
 CONSTRAINT pkMoneda_Id PRIMARY KEY (Id),
 Moneda nvarchar(100) NOT NULL,
 Sigla nvarchar(5) NOT NULL,
 Simbolo nvarchar(5) NULL,
```

```
 Emisor nvarchar(100) NULL,
 Imagen Image NULL
 )

/* Crear indice para MONEDA
 ordenado por MONEDA */
CREATE UNIQUE INDEX ixMoneda
 ON Moneda(Moneda)

/* Crear tabla CAMBIOMONEDA */
CREATE TABLE CambioMoneda(
 IdMoneda int NOT NULL,
 CONSTRAINT fkCambioMoneda_IdMoneda FOREIGN KEY (IdMoneda)
       REFERENCES Moneda(Id),
 Fecha smalldatetime NOT NULL,
 Cambio float NOT NULL
)

/* Crear indice para CAMBIOMONEDA
 ordenado por MONEDA, FECHA */
CREATE UNIQUE INDEX ixCambioMoneda
 ON CambioMoneda(IdMoneda, Fecha)

/* Crear tabla PAIS */
CREATE TABLE Pais(
 Id int identity(1,1) not null,
 CONSTRAINT  pkPais  primary key (Id),
 Pais varchar(50) not null,
 CodigoAlfa2 varchar(5) not null,
 CodigoAlfa3 varchar(5) not null,
 IdMoneda int NOT NULL,
 CONSTRAINT fkPais_IdMoneda FOREIGN KEY (IdMoneda)
       REFERENCES Moneda(Id),
 Mapa image NULL,
 Bandera image NULL
 )

/* Crear indice para PAIS
 ordenado por PAIS */
CREATE UNIQUE INDEX ixPais
 ON Pais(Pais)
```

En estas instrucciones se pueden destacar:

- La instrucción CREATE DATABASE para crear la base de datos
- La instrucción CREATE TABLE para crear una tabla
- La instrucción CREATE INDEX para crear un índice a una tabla

Como se puede observar, todas comienzan con la palabra CREATE seguido del objeto a crear. Como esta instrucción se tienen también ALTER (para modificar) y DROP (para eliminar)

- El Lenguaje de Manejo de Datos (en inglés DML, ***Data Manipulation Language***) permite a los usuarios manejar o tener acceso a los datos que estén organizados por medio del modelo apropiado.

  Este lenguaje es un subconjunto del lenguaje **SQL** que se tratará aquí de una manera no exhaustiva. Este lenguaje se introdujo como lenguaje de consulta del **Sistema R**. La sigla SQL significa ***Structured Query Language*** (*Lenguaje de Consulta Estructurado)*.
  Las siguientes son las instrucciones básicas que trataremos:

  Consulta Básica de Selección
  La estructura básica de una expresión de consulta en SQL se compone de tres cláusulas:

  - La cláusula **Select** que corresponde a la operación de proyección del álgebra relacional. Sirve para listar los atributos que se desean en el resultado de una consulta.
  - La cláusula **From** es una lista de las tablas que se van a examinar durante la ejecución de la expresión.
  - La cláusula **Where** corresponde al predicado de selección del álgebra relacional. Se compone de un predicado que incluye atributos de las tablas que aparecen en la cláusula ***From***.

  Una consulta común en SQL tiene la forma:

```
Select <atributo 1>,<atributo 2>,... ,<atributo n>
from <tabla 1>,<tabla 2>,... ,<tabla n>
Where <predicado>
```

  Esta consulta es equivalente a la expresión del álgebra relacional:

$$\prod_{atributo1,atributo2,\ldots,atributon} \left(\sigma_{predicado}(tabla1\ X\ tabla2\ X\ tablan)\right)$$

  El SQL forma el producto cartesiano de las tablas que se nombran en la cláusula ***From***; realiza una selección del álgebra relacional utilizando el predicado de la cláusula ***Where*** y proyecta el resultado a los atributos de la cláusula ***Select***.

  Si se omite la cláusula ***where*** el predicado *<predicado>* es verdadero.
  La lista de atributos *<atributo 1>,<atributo 2>,... ,<atributo n>* puede sustituirse por un asterisco (*) para elegir todos los atributos de las tablas que aparecen en la cláusula ***From***.

El resultado de una consulta en SQL es una tabla.

Para la eliminación de duplicados de tuplas, se inserta la palabra reservada ***distinct*** después de ***Select*** :

```
Select distinct <atributo 1>,<atributo 2>,... ,<atributo n>
from <tabla 1>,<tabla 2>,... ,<tabla n>
where <predicado>
```

El SQL incluye las operaciones ***Union*** (Unión), ***Intersect*** (Intersección) y ***Minus*** (Diferencia) que operan sobre las tablas.

El SQL utiliza los conectivos lógicos ***and*** (y), ***or*** (o) y ***not*** (no)

El siguiente ejemplo ilustra un ejemplo de consulta *Select* que permite obtener la lista de países que tienen la moneda '*Euro*':

```
SELECT P.Pais , P.CodigoAlfa2
    FROM Pais P
        JOIN Moneda M ON P.IdMoneda = M.Id
    WHERE M.Moneda='Euro';
```

Obsérvese el uso de la cláusula ***Join*** que permite especificar la relación que hay entre 2 tablas

Cuyo resultado es el siguiente:

Pais	CodigoAlfa2
Andorra	AD
Austria	AT
Bélgica	BE
Mayotte	YT
Chipre	CY
Dinamarca	DK
Estonia	EE
Finlandia	FI
Åland	AX
Francia	FR
Guayana Francesa	GF
Tierras Australes y Antárticas Francesas	TF
Alemania	DE
Grecia	GR
Guadalupe	GP
Vaticano, Ciudad del	VA
Irlanda	IE
Italia	IT
Letonia	LV
Lituania	LT
Luxemburgo	LU
Malta	MT
Martinica	MQ
Mónaco	MC
Montenegro	ME
Países Bajos	NL
Portugal	PT
Reunión	RE
San Bartolomé	BL
San Martín	MF

SQL ofrece al usuario cierto control sobre el orden en que se van a mostrar las tuplas. La cláusula ***Order by*** permite que las tuplas en el resultado aparezcan ordenadas por determinados atributos.

Ejemplo:
Supóngase que la consulta anterior se requiere ordenada por el nombre del pais.
La expresión SQL sería:

```
SELECT P.Pais , P.CodigoAlfa2
    FROM Pais P
        JOIN Moneda M ON P.IdMoneda = M.Id
    WHERE M.Moneda='Euro'
    ORDER BY P.Pais;
```

Donde vemos que el resultado varía un poco:

Pais	CodigoAlfa2
Åland	AX
Alemania	DE
Andorra	AD
Austria	AT
Bélgica	BE
Chipre	CY
Dinamarca	DK
Eslovaquia	SK
Eslovenia	SI
España	ES
Estonia	EE
Finlandia	FI
Francia	FR
Grecia	GR
Guadalupe	GP
Guayana Francesa	GF
Irlanda	IE
Italia	IT
Letonia	LV
Lituania	LT
Luxemburgo	LU
Malta	MT
Martinica	MQ
Mayotte	YT
Mónaco	MC
Montenegro	ME
Países Bajos	NL
Portugal	PT
Reunión	RE
San Bartolomé	BL
San Marino	SM

Actualización en SQL

Hasta este momento sólo se ha hecho énfasis en la extracción de información de la base de datos, pero no se ha mencionado ninguna operación que permita añadir, modificar o eliminar información, operaciones conocidas como de actualización.

1. **Inserción**

   Para insertar datos en una tabla debe especificarse la tupla que se va a insertar o una consulta cuyo resultado sea el conjunto de tuplas que se insertará. Esto se denota:

```
Insert into <tabla>
(<atributo 1>, < atributo 2>, ..., < atributo n>)
```

```
values (<valor 1>, <valor 2>, ..., <valor n>)
```

ó

```
Insert into <tabla>
Select ..
```

Ejemplo:

Para agregar el registro de la moneda 'Peso Colombiano' se tendría la siguiente instrucción:

```
INSERT INTO Moneda (Id, Sigla, Moneda)
VALUES(35, 'COP', 'Peso colombiano');
```

2. **Modificación**
Permite cambiar los valores de los atributos de todas o algunas tuplas de una tabla. Se denota:

```
Update <tabla>
set <atributo> = <expresion>
where <predicado>
```

Ejemplo:
Supóngase que en la tabla de la consulta anterior se desea modificar el *Emisor* de la moneda, el cual no había sido insertado (razón por lo cual quedó con valor nulo):

```
UPDATE Moneda
    SET Emisor='Banco de la República'
    WHERE Id=35;
```

3. **Eliminación**
Funciona de manera similar a una consulta, sólo que el resultado es la desaparición de las tuplas elegidas:

```
Delete from <tabla>
where <predicado>
```

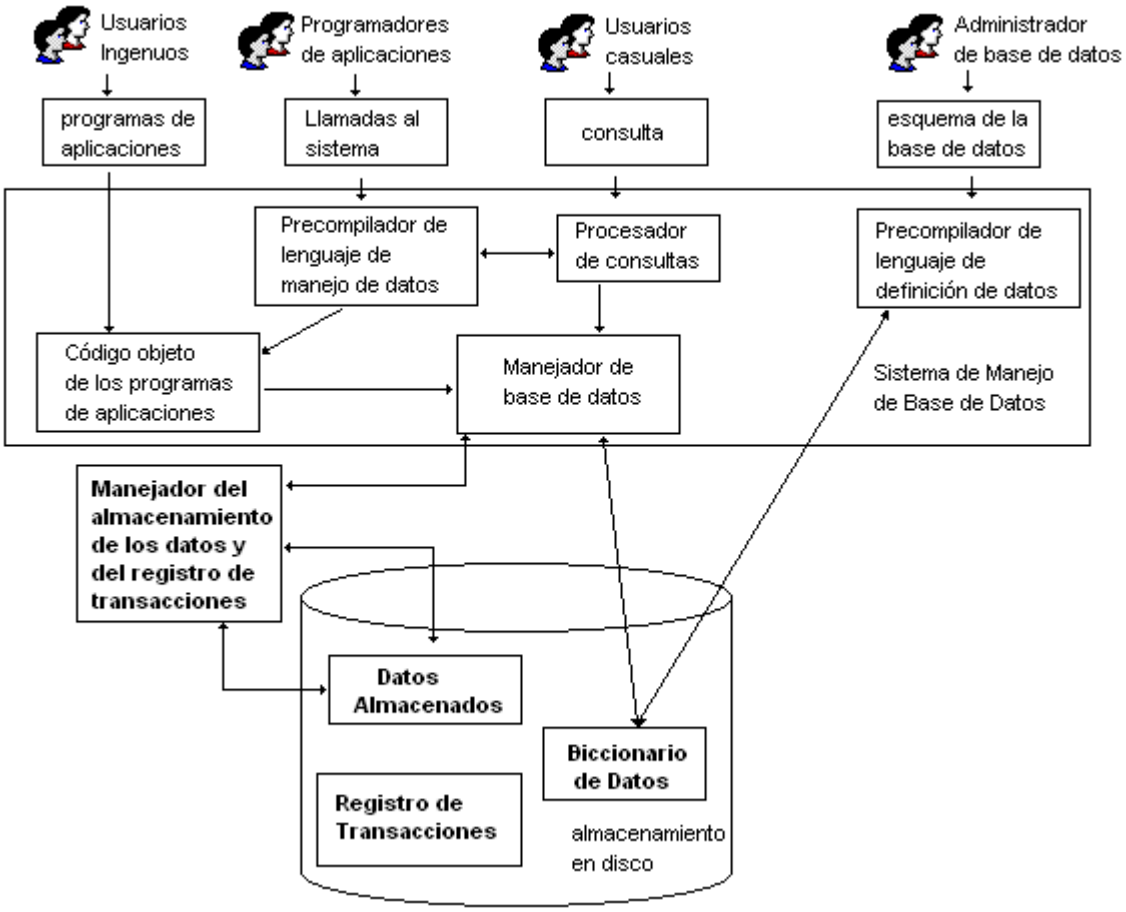
Es de anotar que este comando, como los anteriores, sólo operan sobre una tabla.

Ejemplo:
Para la tabla de la consulta anterior, eliminar la moneda con *Sigla='COU'*

```
DELETE FROM Moneda
    WHERE Sigla='COU';
```

- El **Manejador de Base de Datos** (En inglés ***DataBase Management System***) es un módulo de programa que constituye la interfaz entre los datos de bajo nivel almacenados en la base de datos y los programas de aplicaciones y las consultas hechas al sistema. Sus tareas básicas son:
  - Interacción con el manejador de archivos. Es quién se encarga realmente del almacenamiento, recuperación y actualización de los datos en la base de datos.
  - Implantación de la integridad. Realiza las operaciones que limitan la consistencia de los datos.
  - Puesta en práctica de la seguridad. Hace cumplir los requisitos de seguridad para tener acceso a los datos.
  - Respaldo y recuperación. Es responsable de detectar fallas y restaurar la base de datos al estado que existía antes presentarse ésta.
  - Control de Concurrencia. Vela por la consistencia de la información cuando varios usuarios actualizan a la vez la base de datos.

Un sistema de base de datos se divide en una serie de módulos que se encargan de cada una de las tareas del sistema general. El siguiente diagrama describe sus componentes básicos y cómo interactúan los diferentes tipos de usuarios:



La persona que ejerce un control centralizado sobre un DBMS se le conoce como el Administrador de Base de Datos (en inglés DBA, ***Data Base Administrator***). Sus funciones principales son:

- Definición de Esquema. Es el creador del esquema original de la base de datos en un DDL.
- Modificación del esquema y de la organización física.
- Administración de la seguridad. Concede autorizaciones para el acceso a los datos a los distintos usuarios de la base de datos.
- Especificación de las limitantes de Consistencia.

¿Qué es SQL Server?

SQL Server es un sistema de administración de bases de datos relacional desarrollado por Microsoft. Se apoya en ***Transact-SQL*** (T-SQL), un lenguaje de consulta que extiende el estándar SQL y permite la creación de procedimientos almacenados, funciones y triggers para gestionar consultas complejas y operaciones de bases de datos.

Aunque SQL Server es un producto comercial, Microsoft ofrece ediciones gratuitas, como ***SQL Server Express***, lo que permite a desarrolladores y pequeñas empresas aprovechar sus potentes funcionalidades sin costo alguno. Con el respaldo de Microsoft, SQL Server se ha consolidado como una opción confiable y robusta en entornos empresariales.

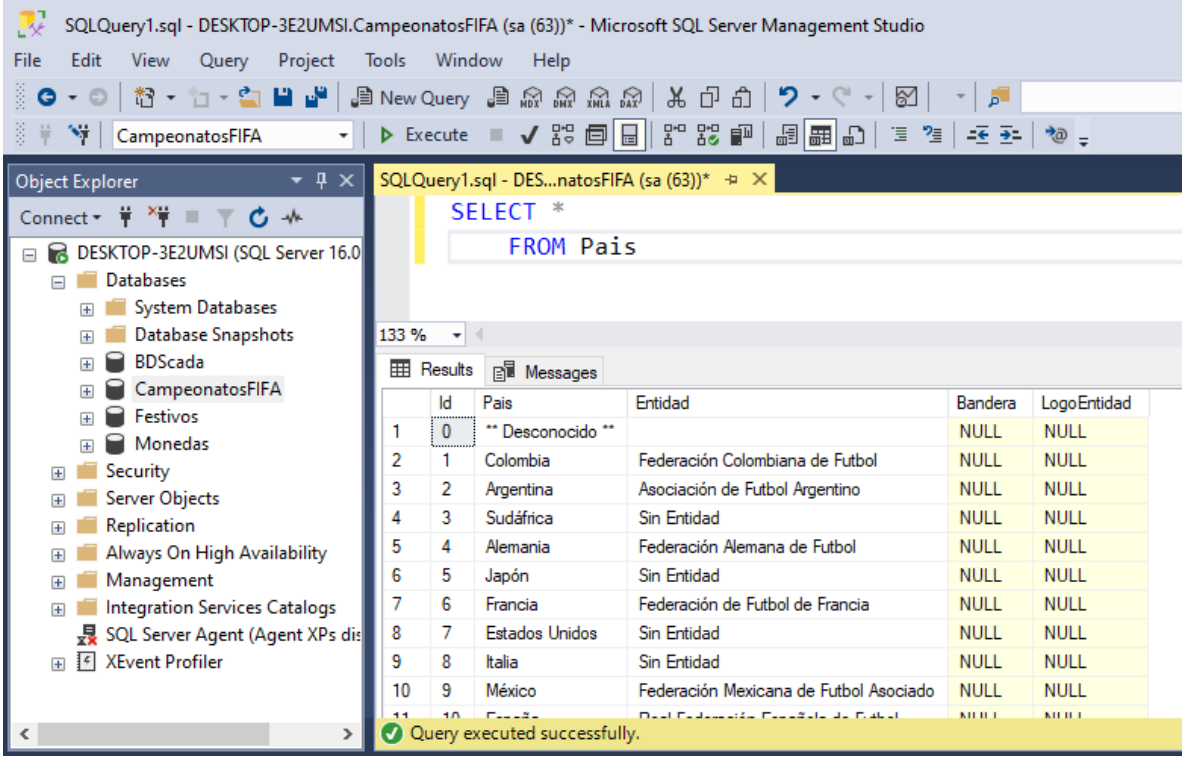
Entre sus características destacadas se encuentran:

- **Rendimiento y Escalabilidad**: SQL Server incluye funcionalidades avanzadas de optimización, como índices, particionamiento de tablas y capacidades in-memory, lo que permite gestionar grandes volúmenes de datos con alto rendimiento.
- **Seguridad y Cumplimiento ACID**: Proporciona sólidas medidas de seguridad y garantiza el cumplimiento de los principios ACID (atomicidad, consistencia, aislamiento y durabilidad) en las transacciones, asegurando la integridad y fiabilidad de los datos incluso ante fallos.
- **Integración y Herramientas Avanzadas**: La estrecha integración con el ecosistema de Microsoft (incluido .NET) facilita el desarrollo de aplicaciones empresariales. Además, cuenta con herramientas de administración y análisis de datos, como ***SQL Server Management Studio*** (SSMS) y **Azure Data Studio**, que permiten desde tareas de mantenimiento básico hasta la implementación de soluciones de inteligencia de negocios (a través de servicios como ***Integration Services***, ***Analysis Services*** y ***Reporting Services***).

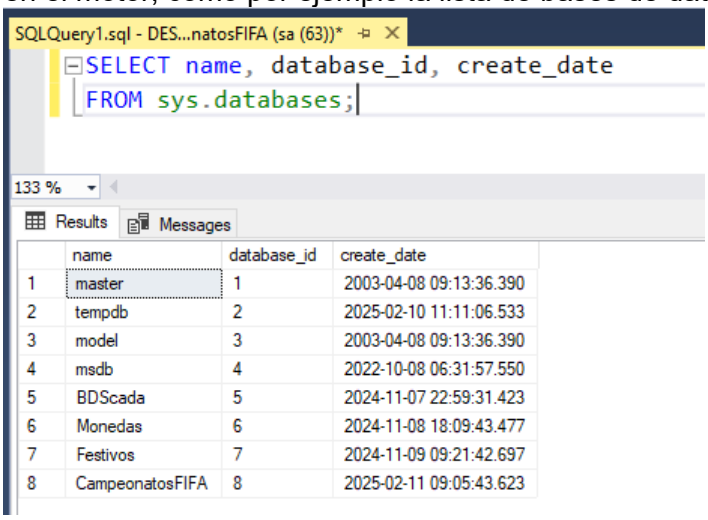
En resumen, SQL Server se destaca por su robustez, escalabilidad y un amplio conjunto de herramientas que facilitan tanto la administración como el análisis de datos, lo que lo convierte en una opción preferida para organizaciones que requieren un sistema seguro y de alto rendimiento para gestionar sus bases de datos.

Management Studio (SSMS)

SQL Server Management Studio (SSMS) es la herramienta oficial de Microsoft diseñada para la administración, desarrollo y supervisión de bases de datos SQL Server. Con una interfaz gráfica intuitiva, SSMS permite a administradores y desarrolladores ejecutar consultas, configurar servidores, gestionar la seguridad y optimizar el rendimiento de sus bases de datos de forma centralizada. Además, incorpora potentes funcionalidades como un editor de código con resaltado de sintaxis, herramientas de depuración y análisis de rendimiento, lo que lo convierte en un entorno integral para facilitar tanto tareas rutinarias como operaciones complejas dentro de infraestructuras empresariales.



Es posible consultar en la base de datos del sistema, los diferentes objetos que se creen en el motor, como por ejemplo la lista de bases de datos:



Cada vez que se cree o cambie la estructura de una tabla, es posible verificar su nueva estructura mediante la siguiente consulta:

SQLQuery1.sql - DES...natosFIFA (sa (63))*

```sql
SELECT
    c.name AS ColumnName,
    t.name AS DataType,
    c.max_length,
    c.is_nullable
FROM sys.columns c
INNER JOIN sys.types t ON c.user_type_id = t.user_type_id
INNER JOIN sys.tables tb ON c.object_id = tb.object_id
WHERE tb.name = 'Estadio'
```

121 %

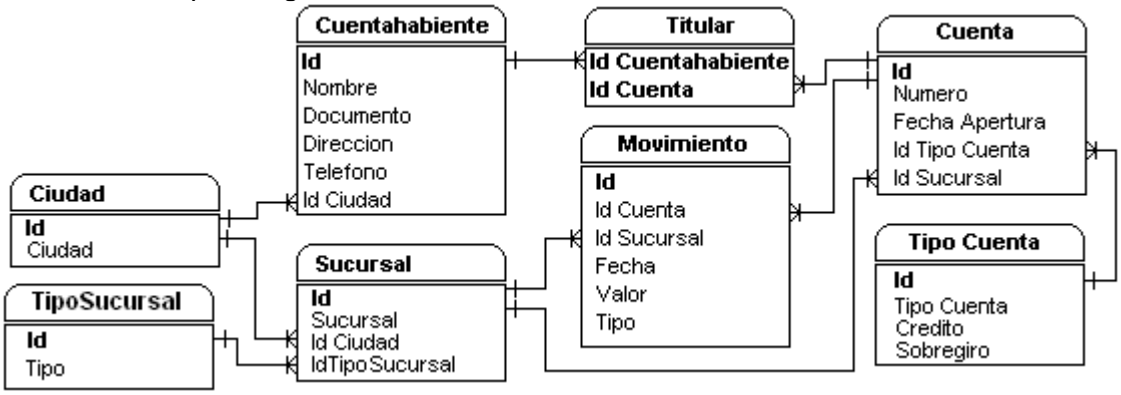
Results | Messages

	ColumnName	DataType	max_length	is_nullable
1	Id	int	4	0
2	Estadio	varchar	100	0
3	IdCiudad	int	4	0
4	Capacidad	int	4	0
5	Foto	image	16	1

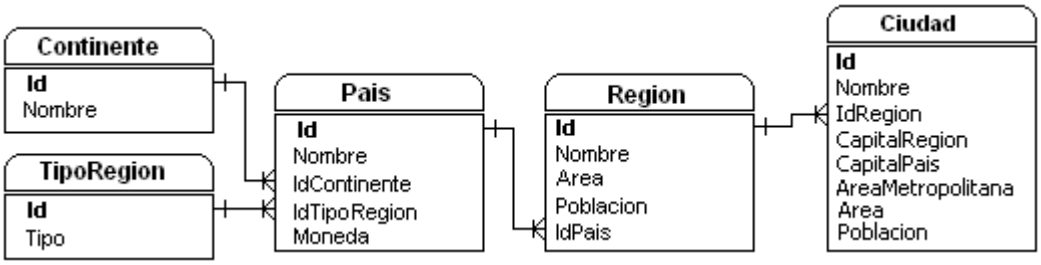
Ejercicios de repaso:

Basado en cada uno de los siguientes modelos relacionales, escribir las instrucciones DDL necesarias para implementar el modelo completo en el motor de base de datos PostgreSQL.

1. Base de datos para registro de cuentas bancarias:



2. Basado en el modelo relacional de división política mundial, escribir las instrucciones SQL necesarias para realizar las consultas que se enuncian, en el motor de base de datos *SQL Server*



a. ¿Cuántas divisiones políticas tiene un país y como se le denomina a la división?
b. ¿Cuál es el área y población de los países del continente “América”?
c. ¿Cuál es la capital del país “Argentina”?
d. ¿Dónde está ubicada la ciudad con nombre “Barbosa”?
e. ¿De qué región es capital la ciudad de “Hermosillo”?
f. ¿Cuál es la región de “Brasil” con mayor población?
g. ¿Cuál es la región de “México” con menor área?
h. ¿De qué país es capital “Bogotá”? (El nombre completo de la ciudad es “SANTA FE DE BOGOTA D.C.”)

En las expresiones de consulta, es bueno tener en cuenta la siguiente lista de funciones:

Tipo	**Función**	**Descripción**
Texto	**upper**(s), **lower**(s)	Convierte una variable de texto s a mayúscula o minúscula
	ltrim(s), **rtrim**(s)	Quita los espacios en blanco a la izquierda o la derecha del texto s
	substring(s,p,l)	Extrae un texto de longitud l desde el texto s a partir de la posición p
	charindex(s2,s1)	Devuelve la posición de la cadena s2 en la cadena s1
	len(s)	Longitud del texto s
Fecha	**getdate()**	Fecha actual del servidor
Numérica	**round**(x,p)	Redondea el número real x según la precisión p
	abs(x)	Valor absoluto de x
	rand()	Genera un numero aleatorio
Conversión	**convert** (tipo_de_dato, expresion)	Convierte a tipo_de_dato una expresion

Y lo siguientes operadores

Operador	**Descripción**
+ - / *	Aritmético
=	Igualdad
<>	Diferente
+	Concatenación de textos

R/

a. Incluyendo todos los países que no tienen regiones registradas:

```
SELECT P.Nombre, COUNT(R.Id), T.Tipo
    FROM Region R RIGHT JOIN Pais P ON R.IdPais = P.ID
INNER JOIN TipoRegion T ON P.IdTipoRegion = T.Id
    GROUP BY P.Nombre, T.Tipo
    ORDER BY P.Nombre
```

Incluyendo sólo los países que tienen regiones registradas:

```
SELECT P.Nombre, COUNT(R.Id), T.Tipo
    FROM Region R INNER JOIN Pais P ON R.IdPais = P.ID
INNER JOIN TipoRegion T ON P.IdTipoRegion = T.Id
    GROUP BY P.Nombre, T.Tipo
    ORDER BY P.Nombre
```

b.

```
SELECT P.Nombre, SUM(R.Area) AS TotalArea,
SUM(R.Poblacion) AS TotalPoblacion
    FROM Region R INNER JOIN Pais P ON R.IdPais = P.ID
INNER JOIN Continente C ON P.IdContinente = C.Id
    WHERE UPPER(C.Nombre)='AMERICA'
    GROUP BY P.Nombre
    ORDER BY P.Nombre
```

c.

```
SELECT C.Nombre
     FROM Pais P INNER JOIN Region R ON P.Id = R.IdPais
INNER JOIN Ciudad C ON R.Id = C.IdRegion
     WHERE LOWER(P.Nombre)='argentina'
     AND C.CapitalPais=true
```

d.

```
SELECT R.Nombre + ' (' + P.Nombre + ')' AS Ubicacion
     FROM Pais P INNER JOIN Region R ON P.Id = R.IdPais
INNER JOIN Ciudad C ON R.Id = C.IdRegion
     WHERE LOWER(C.Nombre)='barbosa'
     ORDER BY R.Nombre, P.Nombre
```

e.

```
SELECT R.Nombre
    FROM Region R
        JOIN Ciudad C
            ON R.Id=C.IdRegion
        WHERE C.Nombre='Hermosillo' AND
C.CapitalRegion=true
```

f.

```
SELECT Region.nombre, Region.Poblacion
    FROM Region
    WHERE Poblacion IN
        (SELECT MAX(Region.Poblacion)
            FROM Pais INNER JOIN Region ON
Pais.Id=Region.IdPais
            WHERE Pais.Nombre='BRASIL')
```

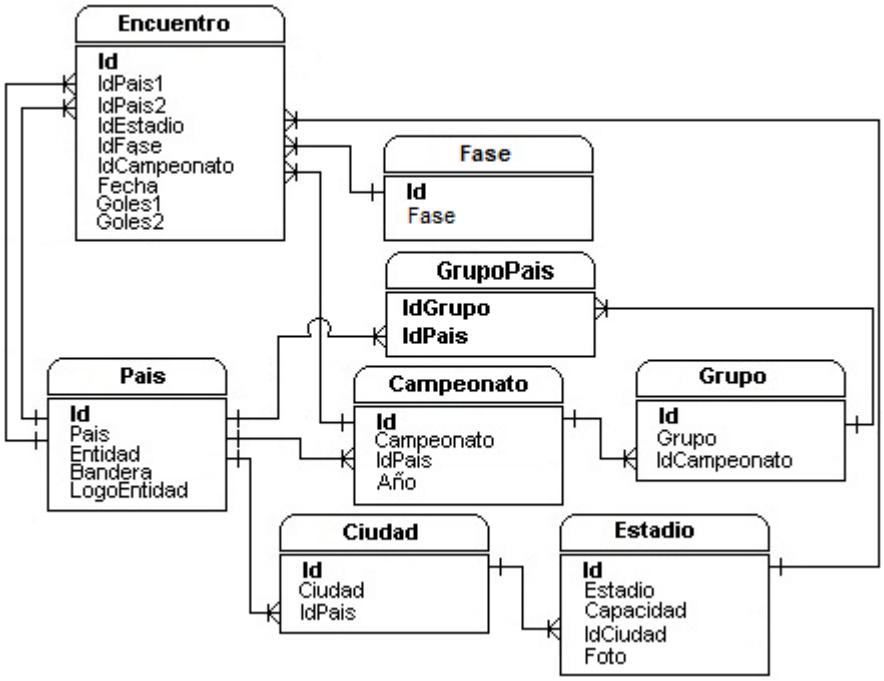
g.

```
SELECT Region.Nombre, Region.Area
    FROM Pais INNER JOIN Region ON Pais.Id=Region.IdPais
    WHERE Pais.Nombre='MEXICO'
    AND Area IN
        (SELECT MIN(Region.Area)
            FROM Pais INNER JOIN Region ON
Pais.Id=Region.IdPais
            WHERE Pais.Nombre='MEXICO')
```

h.

```
SELECT P.Nombre
     FROM Pais P INNER JOIN Region R ON P.Id = R.IdPais
INNER JOIN Ciudad C ON R.Id = C.IdRegion
     WHERE POSITION('BOGOTA' IN UPPER(c.Nombre))>0
     AND C.CapitalPais=true
```

3. Basado en el siguiente modelo relacional para registro de campeonatos de fútbol de selecciones:



escribir las instrucciones SQL necesarias para realizar las consultas que se enuncian, en el motor de base de datos *PostgreSQL:*

a. Puntos obtenidos por un equipo en un grupo
b. ¿Equipo que más participaciones ha tenido en campeonatos?
c. ¿Equipo que más campeonatos ha ganado?
   Tenga en cuenta que una de las fases es la FINAL
d. Ciudades que tengan más de 1 estadio
e. Campeonato en que se haya marcado más goles

R/

a.

```
SELECT (SELECT Pais FROM Pais WHERE Id=@IdPais),
    (SELECT Sum(CASE WHEN (IdPais1=@IdPais AND Goles1>Goles2)
        OR (IdPais2=@IdPais AND Goles1<Goles2)
        THEN 1 ELSE 0 END)
        FROM Encuentro INNER JOIN Grupo
            ON Encuentro.IdCampeonato=Grupo.IdCampeonato
        WHERE (IdPais1=@IdPais OR IdPais2=@IdPais)
        AND IdFase=1 AND Grupo.Id=@IdGrupo
        AND Goles1 IS NOT NULL
        AND Goles2 IS NOT NULL)*3+
    (SELECT Sum(CASE WHEN (IdPais1=@IdPais AND Goles1=Goles2)
        OR (IdPais2=@IdPais AND Goles1=Goles2)
        THEN 1 ELSE 0 END)
        FROM Encuentro INNER JOIN Grupo
```

```
            ON Encuentro.IdCampeonato=Grupo.IdCampeonato
        WHERE (IdPais1=@IdPais OR IdPais2=@IdPais)
        AND IdFase=1 AND Grupo.Id=@IdGrupo
        AND Goles1 IS NOT NULL
        AND Goles2 IS NOT NULL) AS Puntos
```

b. ¿Equipo que más participaciones ha tenido en campeonatos?

```
SELECT TOP 1 Pais, COUNT(IdPais) AS Participaciones
    FROM Pais INNER JOIN GrupoPais
        ON Pais.Id=GrupoPais.IdPais
    GROUP BY Pais
    ORDER BY Participaciones DESC
```

c. ¿Equipo que más campeonatos ha ganado?
Tenga en cuenta que una de las fases es la FINAL

```
SELECT TOP 1 Pais, ((SELECT COUNT(IdPais1)
        FROM Encuentro
        WHERE IdPais1=P.Id AND Goles1>Goles2
        AND IdFase=5
        AND Goles1 IS NOT NULL
        AND Goles2 IS NOT NULL)+
        (SELECT COUNT(IdPais2)
        FROM Encuentro
        WHERE IdPais2=P.Id AND Goles2>Goles1
        AND IdFase=5
        AND Goles1 IS NOT NULL
        AND Goles2 IS NOT NULL)) AS PG
    FROM Pais P
        WHERE Id IN (SELECT IdPais1 AS IdPais
                        FROM Encuentro
                        WHERE IdFase=5
                    UNION SELECT IdPais2 AS IdPais
                        FROM Encuentro
                        WHERE IdFase=5)
    ORDER BY PG DESC
```

Se asume que la fase *"Final"* tiene Id=5

d. Ciudades que tengan más de 1 estadio

```
SELECT Ciudad
    FROM Ciudad
    WHERE Id IN (SELECT IdCiudad
                    FROM Estadio
                        GROUP BY IdCiudad
                    HAVING Count(*) > 1)
```

e. Campeonato en que se haya marcado más goles

```
SELECT TOP 1 Campeonato, SUM(Goles1)+SUM(Goles2) AS Goles
    FROM Encuentro INNER JOIN Campeonato
        ON Encuentro.IdCampeonato=Campeonato.Id
    GROUP BY Campeonato
    ORDER BY Goles DESC
```